

Client-Server, REST APIs, API Gateway, Load Balancer, Reverse Proxy

1. Overview

This topic explains how modern backend systems receive, route, protect, scale, and respond to client requests.

Definition

- **Client-server architecture** is a model where a client, such as a browser or mobile app, sends requests to a server, and the server processes them and returns responses.
- **REST APIs** are a common way for clients and servers to communicate using HTTP methods like **GET**, **POST**, **PUT**, **PATCH**, and **DELETE**.
- **API gateway** is a front-door service that manages API traffic before requests reach backend services.
- **Load balancer** distributes incoming traffic across multiple servers so no single server gets overloaded.
- **Reverse proxy** sits in front of servers and forwards client requests to the correct backend while hiding internal server details.

Why It Matters

Most real-world applications are not a single program running on one machine. They involve browsers, mobile apps, backend servers, databases, caches, authentication systems, and third-party services. These concepts explain how requests move safely and efficiently through that system.

Where It Is Used In Real Systems

- Web applications like Amazon, Flipkart, Gmail, and LinkedIn
- Mobile apps calling backend APIs
- Microservice architectures
- Cloud deployments using AWS, Azure, or GCP
- Content delivery, authentication, rate limiting, and traffic routing
- Backend systems handling high traffic

Why Interviewers Ask About It

Interviewers ask this because it tests whether you understand how real applications work beyond writing code. It checks your knowledge of HTTP, scaling, APIs, request routing, reliability, and system design trade-offs.

2. Core Idea

The core idea is simple:

A client sends a request. The system routes that request through infrastructure components. A backend service processes it and returns a response.

Intuition

Think of a large hospital.

- The **patient** is the client.
- The **reception desk** is the API gateway.
- The **queue manager** is the load balancer.
- The **internal hospital routing system** is the reverse proxy.
- The **doctor** is the backend service.
- The **medical record system** is the database.

The patient does not directly search for the correct doctor. The hospital has a front desk, routing rules, queues, and departments. Similarly, clients do not usually talk directly to internal servers.

Small Example

A user opens a shopping app and clicks "View Cart".

```
Mobile App
  |
  | GET /cart
  v
API Gateway
  |
  | validates token, checks rate limit
  v
Load Balancer
  |
  | chooses one healthy cart server
  v
Cart Service
  |
  | fetches cart data
  v
Database / Cache
```

The response comes back through the same path.

Step-By-Step Explanation

1. The client creates an HTTP request.
2. DNS resolves the domain name to an IP address.
3. The request reaches a public entry point.
4. A reverse proxy or API gateway accepts the request.
5. Authentication, rate limiting, logging, or request validation may happen.
6. A load balancer selects a healthy backend server.
7. The backend service handles business logic.
8. The backend may call a database, cache, queue, or another service.
9. The server returns an HTTP response.
10. The client renders or uses the response.

3. Important Subtopics

3.1 Client-Server Architecture

What It Means

Client-server architecture separates the user-facing part from the processing and data storage part. The client requests services, and the server provides them.

Why It Matters

It creates clear separation of responsibilities:

- Client handles user interaction.
- Server handles business logic, security, persistence, and shared state.

Example

In an online food delivery app:

- Client: Android app
- Server: backend service that handles restaurants, orders, payments, and delivery tracking

Common Interview Angle

Interviewers may ask:

- What happens when a browser opens a website?
- Why should business logic usually stay on the server?
- What are the advantages and disadvantages of client-server architecture?

3.2 HTTP Request-Response Model

What It Means

HTTP is a stateless protocol where the client sends a request and the server sends a response.

Why It Matters

REST APIs, browsers, gateways, proxies, and load balancers mostly work around HTTP behavior.

Example

```
GET /users/42 HTTP/1.1
Host: api.example.com
Authorization: Bearer token
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 42,
  "name": "Asha"
}
```

Common Interview Angle

Interviewers expect you to know:

- HTTP methods
- Status codes
- Headers
- Request body vs response body
- Statelessness

3.3 REST APIs

What It Means

REST is an architectural style for designing APIs around resources. Resources are identified using URLs, and actions are represented using HTTP methods.

Why It Matters

REST makes APIs predictable, scalable, and easy to consume.

Example

GET	/users	-> list users
GET	/users/10	-> get user 10
POST	/users	-> create user
PUT	/users/10	-> replace user 10
PATCH	/users/10	-> partially update user 10
DELETE	/users/10	-> delete user 10

Common Interview Angle

Interviewers ask about:

- REST principles
- HTTP methods
- Idempotency
- Status codes

- Stateless APIs
- REST vs RPC

3.4 API Gateway

What It Means

An API gateway is a single entry point for API requests. It sits between clients and backend services.

Why It Matters

It centralizes cross-cutting concerns:

- Authentication
- Authorization
- Rate limiting
- Request routing
- API versioning
- Logging
- Monitoring
- Request transformation

Example

```
Client
|
v
API Gateway
|----- /users/* -----> User Service
|----- /orders/* -----> Order Service
|----- /pay/* -----> Payment Service
```

Common Interview Angle

Interviewers may ask:

- Why use an API gateway in microservices?
- Difference between API gateway and load balancer
- What can go wrong if the gateway becomes a bottleneck?

3.5 Load Balancer

What It Means

A load balancer distributes traffic across multiple backend servers.

Why It Matters

It improves:

- Scalability
- Availability
- Fault tolerance
- Performance

Example

```

      +----- Server 1
Client Request -> Load Balancer
      +----- Server 2
      +----- Server 3
```

If Server 2 fails, the load balancer can stop sending traffic to it.

Common Interview Angle

Interviewers ask about:

- Load balancing algorithms
- Health checks
- Sticky sessions
- Layer 4 vs Layer 7 load balancing
- Horizontal scaling

3.6 Reverse Proxy

What It Means

A reverse proxy receives client requests and forwards them to backend servers. The client does not know which internal server handled the request.

Why It Matters

It provides:

- Server hiding
- SSL termination
- Caching
- Compression
- Routing
- Security filtering

Example

Nginx can act as a reverse proxy:

```
Client -> Nginx Reverse Proxy -> Backend App Server
```

Common Interview Angle

Interviewers ask:

- Forward proxy vs reverse proxy
- Reverse proxy vs load balancer
- Why use Nginx before application servers?

3.7 Forward Proxy

What It Means

A forward proxy sits in front of clients and sends requests on their behalf.

Why It Matters

It is used for:

- Client anonymity
- Access control
- Content filtering
- Corporate network monitoring

Example

In a college network, students may access the internet through a proxy server.

Common Interview Angle

Interviewers often compare forward proxy and reverse proxy.

3.8 SSL/TLS Termination

What It Means

SSL termination means decrypting HTTPS traffic at a gateway, reverse proxy, or load balancer before forwarding it internally.

Why It Matters

It reduces work on backend servers and centralizes certificate management.

Example

```
Client -- HTTPS --> Load Balancer -- HTTP or HTTPS --> Backend Server
```

Common Interview Angle

Interviewers may ask whether internal traffic should still be encrypted. In high-security systems, the answer is usually yes.

3.9 Rate Limiting

What It Means

Rate limiting restricts how many requests a client can make in a time window.

Why It Matters

It protects systems from abuse, accidental overload, brute-force attacks, and unfair usage.

Example

```
Maximum 100 requests per user per minute
```

Common Interview Angle

Interviewers may ask where to implement rate limiting. Common answers are API gateway, reverse proxy, or a dedicated distributed rate limiter using Redis.

3.10 Health Checks

What It Means

Health checks are periodic checks used by load balancers or orchestrators to decide whether a server is healthy.

Why It Matters

They prevent traffic from going to failed or unhealthy servers.

Example

```
GET /health  
  
Response: 200 OK
```

Common Interview Angle

Interviewers may ask the difference between shallow health checks and deep health checks.

4. Real-World Example

Opening A Product Page On An E-Commerce Website

Suppose a user opens:

```
https://shop.example.com/products/123
```

Possible flow:

```
Browser
|
| DNS lookup for shop.example.com
v
CDN / Edge Server
|
| cache static files if available
v
Reverse Proxy
|
| SSL termination, compression, routing
v
API Gateway
|
| auth check, rate limit, request logging
v
Load Balancer
|
| choose a healthy product-service instance
v
Product Service
|
| query product data
v
Cache / Database
```

The product service returns product details. The response travels back to the browser. The browser then renders HTML, CSS, JavaScript, and images.

Backend Server Example

A Node.js, Java Spring Boot, Django, or Go server may expose REST endpoints:

```
GET /products/123
POST /cart/items
```

```
POST /orders
GET /orders/999
```

In production, this server usually runs behind a reverse proxy, gateway, or load balancer.

Distributed System Example

In a microservice system:

```
Client -> API Gateway -> Load Balancer -> Service A -> Service B -> Database
```

The gateway handles external concerns. The services focus on business logic.

5. Diagrams / Mental Models

Basic Client-Server Model

```
+-----+      Request      +-----+
| Client | -----> | Server |
|        | <-----         |        |
+-----+      Response     +-----+
```

REST Resource Model

Resource: User

```
GET    /users/1    read
POST   /users      create
PUT    /users/1    replace
PATCH /users/1    update partly
DELETE /users/1    delete
```

Production Request Path

```
+-----+ +-----+ +-----+ +-----+ +-----+
--+
| Client | -> | API Gateway | -> | Load Balancer | -> | App Server   | -> |
Database|
+-----+ +-----+ +-----+ +-----+ +-----+
--+
|
+-- auth
+-- rate limit
```

```
+-- logging
+-- routing
```

Reverse Proxy Mental Model

Client thinks:

Client -> example.com

Actual system:

```
Client -> Reverse Proxy -> Server 1
                           -> Server 2
                           -> Server 3
```

Layer 4 vs Layer 7 Load Balancing

Layer 4: uses IP and port

Layer 7: understands HTTP path, headers, cookies, method

6. Common Interview Questions

1. What is client-server architecture?

Client-server architecture is a model where clients request services and servers provide responses. The client usually handles presentation, while the server handles business logic, data access, authentication, and shared state.

Key points interviewer expects:

- Separation of client and server responsibilities
- Request-response communication
- Used in web, mobile, and distributed systems

Common mistakes:

- Saying the client and server must be on different physical machines
- Ignoring security and data ownership

2. What happens when you enter a URL in a browser?

The browser checks cache, performs DNS lookup, establishes a TCP connection, performs TLS handshake for HTTPS, sends an HTTP request, receives a response, downloads required resources, and renders the page.

Key points interviewer expects:

- DNS

- TCP/TLS
- HTTP request-response
- Server processing
- Browser rendering

Common mistakes:

- Skipping DNS
- Saying the browser directly talks to the database

3. What is a REST API?

A REST API exposes resources using URLs and performs operations using HTTP methods. It is usually stateless and returns representations such as JSON.

Key points interviewer expects:

- Resources
- HTTP methods
- Statelessness
- Status codes
- JSON/XML responses

Common mistakes:

- Treating REST as just any HTTP API
- Using verbs in URLs unnecessarily, such as `/getUser`

4. What is the difference between `PUT` and `PATCH`?

`PUT` usually replaces an entire resource, while `PATCH` partially updates a resource.

Key points interviewer expects:

- `PUT` is generally idempotent
- `PATCH` modifies selected fields
- Both are used for updates

Common mistakes:

- Saying both are exactly the same
- Saying `PATCH` is always non-idempotent

5. What does stateless mean in REST?

Stateless means each request contains all information needed to process it. The server does not rely on previous client requests stored in server-side session state.

Key points interviewer expects:

- Each request is independent
- Improves scalability

- Authentication often uses tokens or cookies

Common mistakes:

- Thinking stateless means no database state
- Thinking login is impossible in stateless APIs

6. What is an API gateway?

An API gateway is a single entry point for client API requests. It routes requests to backend services and handles common concerns like authentication, rate limiting, logging, monitoring, and API versioning.

Key points interviewer expects:

- Entry point
- Routing
- Cross-cutting concerns
- Useful in microservices

Common mistakes:

- Confusing it with only a load balancer
- Putting all business logic inside the gateway

7. What is a load balancer?

A load balancer distributes incoming traffic across multiple backend servers to improve availability, scalability, and performance.

Key points interviewer expects:

- Traffic distribution
- Health checks
- Horizontal scaling
- Fault tolerance

Common mistakes:

- Saying it only improves speed
- Ignoring server health checks

8. What are common load balancing algorithms?

Common algorithms include round robin, weighted round robin, least connections, least response time, IP hash, and random selection.

Key points interviewer expects:

- Round robin is simple
- Least connections helps when requests have different durations
- IP hash can support session stickiness

Common mistakes:

- Assuming round robin is always best
- Ignoring uneven server capacity

9. What is a reverse proxy?

A reverse proxy sits in front of backend servers and forwards client requests to them. It hides internal servers and can provide SSL termination, caching, compression, and routing.

Key points interviewer expects:

- Server-side proxy
- Hides backend details
- Common examples: Nginx, HAProxy, Envoy

Common mistakes:

- Confusing it with a forward proxy
- Saying it is always the same as a load balancer

10. Difference between forward proxy and reverse proxy?

A forward proxy represents clients when accessing servers. A reverse proxy represents servers when handling client requests.

Key points interviewer expects:

- Forward proxy hides clients
- Reverse proxy hides servers
- Different placement in the network

Common mistakes:

- Defining both as "middle servers" without direction
- Not explaining who is being protected or hidden

11. API gateway vs load balancer?

An API gateway manages API-level concerns like authentication, routing, rate limiting, and transformation. A load balancer mainly distributes traffic among backend instances.

Key points interviewer expects:

- Gateway is API-aware
- Load balancer focuses on distribution and availability
- Some products can do both partially

Common mistakes:

- Saying they are completely unrelated
- Saying one always replaces the other

12. Why do we need multiple backend servers?

Multiple backend servers improve scalability and availability. If one server fails, others can continue serving traffic. If traffic grows, more servers can be added horizontally.

Key points interviewer expects:

- Horizontal scaling
- Fault tolerance
- Higher throughput

Common mistakes:

- Forgetting shared state problems
- Assuming adding servers automatically solves database bottlenecks

13. What is sticky session?

Sticky session means the load balancer sends requests from the same client to the same backend server.

Key points interviewer expects:

- Useful when session data is stored locally
- Can reduce flexibility
- Better alternative is often shared session storage

Common mistakes:

- Thinking sticky sessions are always required
- Ignoring failure of the sticky server

14. What is rate limiting and where is it done?

Rate limiting controls how many requests a client can make in a fixed time. It is commonly implemented at an API gateway, reverse proxy, or distributed service using Redis.

Key points interviewer expects:

- Protects backend
- Prevents abuse
- Requires identity: IP, user ID, API key, or token

Common mistakes:

- Rate limiting only by IP in all cases
- Ignoring distributed deployments

15. What is SSL termination?

SSL termination is the process where HTTPS traffic is decrypted at a load balancer, gateway, or reverse proxy before being forwarded to backend servers.

Key points interviewer expects:

- Centralized certificate handling

- Reduces backend work
- Internal traffic may still need encryption

Common mistakes:

- Thinking HTTPS always terminates only at the app server
- Ignoring security of internal networks

7. Deep-Dive Questions

1. How would you design request flow for a microservice-based e-commerce app?

A good flow is:

```
Client -> CDN -> API Gateway -> Load Balancer -> Microservice -> Cache/Database
```

The CDN serves static content. The gateway handles authentication, rate limiting, logging, and routing. The load balancer distributes traffic across service instances. Each microservice owns its business logic and data access.

Important point: Do not make the gateway a place for all business logic. Keep domain logic inside services.

2. How does a load balancer know a server is unhealthy?

It performs health checks. A simple health check may call `/health` and expect `200 OK`. A deeper health check may verify database connectivity, cache availability, or dependency status.

Trade-off:

- Shallow checks are fast but may miss dependency failures.
- Deep checks are more accurate but can overload dependencies or create false failures.

3. Can a reverse proxy also be a load balancer?

Yes. Tools like Nginx, HAProxy, and Envoy can act as both reverse proxies and load balancers. Conceptually, reverse proxy means accepting client requests on behalf of servers, while load balancing means distributing traffic across multiple servers.

The roles overlap in real products, but the concepts are different.

4. How do you avoid the API gateway becoming a single point of failure?

Run multiple gateway instances behind a load balancer. Use health checks, autoscaling, monitoring, fallback behavior, and multi-zone deployment. Configuration changes should be tested carefully because a bad gateway config can break all APIs.

Key idea: The gateway is a critical entry point, so it must itself be highly available.

5. How does REST handle authentication if it is stateless?

REST can use tokens, such as JWTs or opaque access tokens, sent with each request. The server validates the token and processes the request without relying on server-side session memory.

Example:

```
Authorization: Bearer <token>
```

Stateless does not mean unauthenticated. It means each request carries the required context.

8. Comparison Tables

Client vs Server

Aspect	Client	Server
Main role	Sends requests	Processes requests and sends responses
Examples	Browser, mobile app, desktop app	API server, web server, database server
Owns UI?	Usually yes	Usually no
Owns business logic?	Limited or presentation-specific	Usually yes
Owns sensitive data?	Should avoid storing too much	Usually manages it
Interview trap	Client is not always a browser	Server is not always one machine

REST API vs RPC API

Aspect	REST API	RPC API
Main idea	Resource-oriented	Action/function-oriented
URL style	<code>/users/10</code>	<code>/getUserById</code>
Method usage	Uses HTTP methods meaningfully	Often uses <code>POST</code> for many actions
Good for	Public APIs, CRUD resources	Internal service calls, command-style operations
Example	<code>DELETE /orders/99</code>	<code>cancelOrder(orderId)</code>
Interview trap	REST is not just JSON over HTTP	RPC is not automatically bad

API Gateway vs Load Balancer

Aspect	API Gateway	Load Balancer
Main purpose	API management and routing	Traffic distribution
Works at	Usually Layer 7	Layer 4 or Layer 7
Handles auth?	Commonly yes	Usually not the main role

Aspect	API Gateway	Load Balancer
Handles rate limiting?	Commonly yes	Sometimes, but not always
Knows API paths?	Yes	Layer 7 load balancers can
Example	Kong, AWS API Gateway, Apigee	AWS ALB/NLB, HAProxy, Nginx
Interview trap	Gateway may include load balancing features	Load balancer is not a full API management layer

Reverse Proxy vs Forward Proxy

Aspect	Forward Proxy	Reverse Proxy
Sits near	Client side	Server side
Hides	Clients	Servers
Used by	Browsers, corporate networks	Backend systems, websites
Example use	Access control for employees	Nginx in front of app servers
Client knows target server?	Usually yes	Usually only knows public domain
Server sees	Proxy as requester	Reverse proxy as frontend

Reverse Proxy vs Load Balancer

Aspect	Reverse Proxy	Load Balancer
Main purpose	Forward requests to backend servers	Distribute traffic among instances
Can hide backend?	Yes	Yes
Can cache/compress?	Often yes	Sometimes
Selects among servers?	May do so	Core responsibility
Example	Nginx reverse proxy	HAProxy load balancing
Interview trap	One tool can perform both roles	The concepts are not identical

Layer 4 vs Layer 7 Load Balancer

Aspect	Layer 4 Load Balancer	Layer 7 Load Balancer
Uses	IP, port, TCP/UDP	HTTP method, path, headers, cookies
Speed	Usually faster	More processing
Routing intelligence	Lower	Higher

Aspect	Layer 4 Load Balancer	Layer 7 Load Balancer
Example rule	Send TCP traffic on port 443	Send <code>/api/users</code> to user service
Best for	Raw network traffic, high performance	HTTP-aware routing
Interview trap	Layer 7 gives flexibility but has more overhead	

Common HTTP Methods

Method	Purpose	Has Body?	Idempotent?	Example
GET	Read resource	Usually no	Yes	GET <code>/users/1</code>
POST	Create or trigger action	Yes	Usually no	POST <code>/orders</code>
PUT	Replace resource	Yes	Yes	PUT <code>/users/1</code>
PATCH	Partial update	Yes	Not always	PATCH <code>/users/1</code>
DELETE	Delete resource	Usually no	Yes in REST meaning	DELETE <code>/users/1</code>

Common HTTP Status Codes

Code	Meaning	Example
200 OK	Successful request	User fetched
201 Created	Resource created	Order placed
204 No Content	Success with no body	Delete successful
400 Bad Request	Invalid request	Missing required field
401 Unauthorized	Not authenticated	Missing token
403 Forbidden	Authenticated but not allowed	No admin access
404 Not Found	Resource not found	User ID does not exist
409 Conflict	State conflict	Duplicate unique value
429 Too Many Requests	Rate limit exceeded	Too many API calls
500 Internal Server Error	Server-side failure	Unhandled exception
503 Service Unavailable	Service temporarily unavailable	Backend down

9. Common Mistakes

- Thinking REST means only returning JSON.
- Designing REST URLs with verbs everywhere, such as `/createUser` and `/deleteOrder`.
- Confusing API gateway with load balancer.
- Confusing reverse proxy with forward proxy.
- Assuming one server means one physical machine.

- Forgetting that REST is stateless, not state-free.
- Storing session state only in one backend server and then adding load balancing without thinking.
- Ignoring health checks when discussing load balancers.
- Saying **POST** is always for create and nothing else.
- Saying **PUT** and **PATCH** are the same.
- Assuming internal traffic is always safe after SSL termination.
- Putting business logic inside the API gateway.
- Forgetting rate limiting, logging, monitoring, and timeouts in API design.
- Assuming horizontal scaling solves database bottlenecks automatically.

10. Edge Cases / Special Cases

1. Stateless API With Login

A REST API can still support login. The client sends a token or cookie with each request. The server does not need to remember the previous request in local memory.

2. Sticky Sessions

Sticky sessions can help when session state is stored on one server, but they reduce flexibility. If that server fails, the user's session may be affected. A better design is often shared session storage such as Redis.

3. Load Balancer Does Not Fix Bad Backend Design

If the database is the bottleneck, adding more app servers may not help. It may increase pressure on the database.

4. Gateway As Bottleneck

An API gateway centralizes useful logic, but if it is overloaded or misconfigured, it can affect all services.

5. Reverse Proxy Timeout

Even if the backend eventually responds, the reverse proxy may timeout first and return an error to the client.

6. Idempotency In Payment APIs

Payment APIs often require idempotency keys. If a client retries a payment request, the server should not charge the user twice.

Idempotency-Key: abc-123

7. Caching Dynamic Data

Reverse proxies and CDNs can cache responses, but caching personalized or sensitive data incorrectly can cause security issues.

8. Client IP Behind Proxy

The backend may see the proxy's IP instead of the real client's IP. Headers like `X-Forwarded-For` are often used, but they must be trusted carefully.

9. Layer 7 Routing Needs HTTP Knowledge

A Layer 7 load balancer can route `/users` and `/orders` differently, but it must inspect HTTP data. This adds processing overhead.

10. Retry Storm

If clients aggressively retry failed requests, they may increase system load and make an outage worse. Backoff and jitter are important.

11. How to Explain in Interview

Client-server architecture means clients like browsers or mobile apps send requests to servers that process business logic and return responses. REST APIs are a common HTTP-based way to expose server resources using methods like `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`. In production systems, requests usually pass through infrastructure like an API gateway, reverse proxy, and load balancer. The gateway handles API-level concerns like authentication, routing, rate limiting, and logging. The reverse proxy hides backend servers and can handle SSL termination or caching. The load balancer distributes traffic across healthy server instances to improve scalability and availability.

12. Quick Revision Notes

Key Definitions

- **Client:** Program that sends requests.
- **Server:** Program or system that processes requests and returns responses.
- **REST API:** Resource-based API style using HTTP methods and stateless requests.
- **API Gateway:** API front door for routing, auth, rate limiting, logging, and transformations.
- **Load Balancer:** Distributes traffic across multiple backend instances.
- **Reverse Proxy:** Server-side proxy that forwards requests to backend servers.
- **Forward Proxy:** Client-side proxy that forwards requests on behalf of clients.

Important Points

- REST is about resources, not function names.
- HTTP is stateless, but applications can maintain state using databases, tokens, cookies, and caches.
- Load balancers need health checks.
- API gateways should not contain heavy business logic.
- Reverse proxies hide internal server details.
- Multiple servers improve availability only if shared state is handled correctly.

Common Comparisons

- API gateway vs load balancer
- Forward proxy vs reverse proxy
- `PUT` vs `PATCH`

- Layer 4 vs Layer 7 load balancing
- REST vs RPC
- Client vs server

Must-Remember Facts

- **GET** should not modify server state.
- **POST** is usually not idempotent.
- **PUT** is generally idempotent.
- **429** means too many requests.
- **401** means unauthenticated.
- **403** means authenticated but not allowed.
- Reverse proxy hides servers.
- Forward proxy hides clients.

Interview Traps

- Stateless does not mean no database.
- API gateway and load balancer are related but not the same.
- Reverse proxy and load balancer can be implemented by the same tool.
- Adding servers does not automatically fix database, cache, or network bottlenecks.
- Caching authenticated responses must be handled carefully.

13. Practice Tasks

Task 1: Draw Request Flow

Draw the flow for:

```
User opens https://example.com/orders/123
```

Include:

- Browser
- DNS
- Reverse proxy
- API gateway
- Load balancer
- Order service
- Database

Task 2: Design REST Endpoints

Design REST endpoints for a library system:

- Create a book
- Get a book
- List all books

- Update book title
- Delete a book
- Borrow a book
- Return a book

Think carefully about which actions are resource-based and which are command-like.

Task 3: Identify HTTP Status Codes

Choose the correct status code:

Situation	Status Code
User created successfully	?
Invalid email format	?
User not logged in	?
User logged in but not admin	?
Too many requests	?
Backend temporarily down	?

Task 4: Compare Components

Explain these in your own words:

- API gateway vs load balancer
- Reverse proxy vs forward proxy
- **PUT** vs **PATCH**
- REST vs RPC

Task 5: Simulate Load Balancing

Given three servers:

S1, S2, S3

Distribute 10 requests using round robin:

R1 -> ?
R2 -> ?
...
R10 -> ?

Then repeat using weighted round robin:

```
S1 weight = 2  
S2 weight = 1  
S3 weight = 1
```

Task 6: Think About Failure

Answer:

- What happens if one backend server crashes?
- What happens if the load balancer crashes?
- What happens if the API gateway is misconfigured?
- What happens if the database is slow?

Task 7: Implement A Small REST API

Implement a small REST API in Python using Flask or FastAPI:

```
GET    /tasks  
POST   /tasks  
GET    /tasks/{id}  
PATCH /tasks/{id}  
DELETE /tasks/{id}
```

Add:

- Proper status codes
- Input validation
- Simple in-memory storage

Task 8: Analyze A Real Website

Open browser developer tools and inspect:

- Request method
- Status code
- Response headers
- Cookies
- Cache headers
- API calls made by the page

Task 9: Design Rate Limiting

Design a rate limiter for:

```
100 requests per user per minute
```

Think about:

- Where to store counters
- What key to use
- What happens in multiple gateway instances

Task 10: Explain In 60 Seconds

Practice answering:

Explain how a request reaches a backend server in a production system.

Include:

- Client
- DNS
- Gateway/proxy
- Load balancer
- Backend service
- Database

14. Final Cheat Sheet

Core Definition

Client-server architecture is a communication model where clients send requests and servers process them. REST APIs define a structured HTTP-based way for clients and servers to exchange resource data. API gateways, load balancers, and reverse proxies are infrastructure components that route, protect, scale, and optimize this communication.

Why It Matters

These concepts are the foundation of web apps, mobile backends, microservices, and cloud systems. They explain how real production traffic moves from users to backend services reliably and securely.

Most Asked Questions

- What happens when you open a URL?
- What is REST?
- What is statelessness?
- **PUT** vs **PATCH**
- API gateway vs load balancer
- Forward proxy vs reverse proxy
- Reverse proxy vs load balancer
- Layer 4 vs Layer 7 load balancing
- What is rate limiting?
- What is SSL termination?

Common Comparisons

Comparison	One-Line Difference
Client vs server	Client asks, server answers
REST vs RPC	REST is resource-oriented, RPC is action-oriented
API gateway vs load balancer	Gateway manages APIs, load balancer distributes traffic
Forward proxy vs reverse proxy	Forward proxy hides clients, reverse proxy hides servers
Reverse proxy vs load balancer	Reverse proxy forwards, load balancer distributes
Layer 4 vs Layer 7	Layer 4 uses IP/port, Layer 7 understands HTTP
PUT vs PATCH	PUT replaces, PATCH partially updates

One-Line Interview Answer

A production request usually flows from client to DNS, then through a reverse proxy or API gateway for routing, security, and rate limiting, then through a load balancer to a healthy backend server, which processes the request and returns a response.